

Document 5: Workflow Logic & Automation Specs

System Technical Specification for UI Reactivity, Data Ingestion, and Intelligence Orchestration

1. Frontend Event Triggers & UI State Reactivity

Trigger 1: Global Filter Strip State Changes

- **Event:** `onChange` on State, District, or Scheme Dropdowns.
- **Debounce:** 300ms on text search; immediate on dropdown select.
- **Execution Flow:**
 - a. Synchronize selection into browser URL parameters (`?state=27&fy=2025-2026`).
 - b. Dispatch reactive state change to global store (`activeFilters`).
 - c. Initiate parallel asynchronous API calls: `fetchKpis(filters)`, `fetchGeoJsonChoropleth(filters)`, and `fetchTopAnomalies(filters)`.
 - d. Toggle UI skeleton states to `true` across affected cards.
 - e. Upon response resolution, re-render cards, update choropleth map color bins, and toggle skeleton states to `false`.

Trigger 2: Natural Language Query Submission

- **Event:** `onClick` of `#btn-submit-query` or `Enter` in `#nlp-input-field`.
- **Execution Flow:**
 - a. Validate prompt length (minimum 8 characters) and run local sanitization against known SQL injection keywords.
 - b. Transition response canvas into a shimmering skeleton loading state.
 - c. Dispatch POST request to `/api/v1/intelligence/query-orchestrator` with `{ prompt, current_filters }`.
 - d. On HTTP 200: Stream synthesized narrative summary, render generated SQL into syntax highlighter, populate provenance citations, and mount dynamic chart.
 - e. On HTTP 422: Display validation toast regarding query scope guardrails.

- f. On HTTP 500: Display inline error banner with trace ID.

2. Backend / API Workflows & Automation Architecture

Workflow A: Automated Ingestion & LGD Harmonization (n8n Pipeline)

Parameter	Specification
Schedule	Daily at 02:00 AM IST (Cron: 0 2 * * *)
Target Schemes	MGNREGA, PM-KISAN, PMAY-G
Source API	Open Government Data (data.gov.in)
Staging Table	staging_unstructured_imports

- **Step 1 (Ingestion):** HTTP Request nodes fetch daily CSV/JSON delta releases from Open Government Data endpoints.
- **Step 2 (Staging):** Ingest raw records into `staging_unstructured_imports`.
- **Step 3 (Normalization & Entity Resolution):** Execute database function `fn_harmonize_lgd_entities()`:
 - Match incoming district and block strings against `dim_lgd_geography.entity_name`.
 - Fall back to Levenshtein distance matching (`levenshtein(lower(source), lower(target)) <= 2`).
 - Flag unresolved strings with `lgd_code = 999999` and queue for human review.
- **Step 4 (Upsert):** Write validated records into `fact_scheme_allocations` and `fact_scheme_beneficiaries_agg` using PostgreSQL `ON CONFLICT DO UPDATE`.

Workflow B: Automated Convergence & Anomaly Detection Job

- **Schedule:** Weekly every Monday at 04:00 AM IST (Cron: 0 4 * * 1).
- **Logic Execution:** Executes stored procedure `sp_detect_governance_anomalies()`:
 - a. **Underutilization Flag:** Compute Z-score of fund utilization percentage across all districts within each state:
$$Z = \frac{x_i - \text{mean}}{\text{std_dev}}$$

Where $x_i = \frac{\text{utilized_cr}}{\text{released_cr}}$. If $Z \leq -2.0$ and current fiscal quarter $\geq \text{Q3}$, flag as **CRITICAL : UNDERUTILIZATION**.

- b. **Scheme Overlap & Gap Flag:** Identify intersections where MGNREGA person-days generated are within the top 20th percentile, yet PMAY-G physical completion is in the bottom 20th percentile for the same LGD block over a 90-day window.
- c. **Record Insertion:** Insert flagged records into `fact_detected_anomalies` with computed Z-scores and input variables saved in the `evidence_payload` JSONB column.

Workflow C: Text-to-SQL Natural Language Orchestrator

- **Trigger:** Webhook from WeWeb frontend hitting `/api/v1/intelligence/query-orchestrator`.
- **Execution Flow:**
 - a. **Context Injection:** Inject database schema definition (tables, relationships, enums) and user JWT authorization claims (role, assigned LGD).
 - b. **LLM Generation:** Query LLM with strict prompt: generate read-only PostgreSQL query adhering to user LGD boundary constraints, returning JSON { `sql`, `rationale` }.
 - c. **AST Parser Verification:** Validate generated SQL using an Abstract Syntax Tree (AST) parser to ensure zero write operations (`INSERT`, `UPDATE`, `DELETE`, `DROP`, `TRUNCATE`) and enforce a mandatory `LIMIT 500`.
 - d. **Database Execution:** Execute query on isolated PostgreSQL read-replica using a restricted read-only database user.
 - e. **Second-Stage Synthesizer:** Pass query output to LLM to generate an executive markdown takeaway citing exact columns and underlying record IDs.